A. 四代算子编程抽象的硬件控制粒度与开发效率定位

CUDA C++ / CUTLASS 3.x
~1,800 行 · 100% 硬件控制
手写 mbarrier 与寄存器调度

Triton 3.x（块级隐式编译）
~90 行 · B200 利用率 ~76%
难控 Warp-Group 与 TMEM

ThunderKittens 2.0
~220 行 C++ · 96% 硬件峰值
16×16 瓦片 + NVLink 重叠

TileLang（三级显式流水）
~80 行 Python · 98% 峰值
Kernel + pipelined_for + copy

演进主线：将 TMEM/寄存器布局与异步 DMA 显式暴露给高层原语

B. 算例 5-4 / 实验 5-17：Blackwell TMEM 双缓冲 TMA 流水对比

朴素单缓冲（累加器在寄存器）
TMA 0.75μs + MMA 1.00μs 串行
16 步 28.00 μs（利用率 57.1%）

TileLang 双缓冲 + TMEM
预取 0.75μs + 16 步 16.00μs
总计 16.75 μs（提速 1.67×）

单缓冲：128 regs/线程
256 线程占 32,768 寄存器

TMEM 卸载：56 regs/线程
省 56.2% 寄存器供预取

结论：双缓冲流水 + 256 KB TMEM 驻留同时打破带宽空泡与寄存器墙